# Amalgamation Inspector — Mobile Demo App: Task List

Scope: wrap the existing Flask/Gemini backend with an Expo (React Native) Android app, reachable via ngrok, supporting both `with_references` and `prompt_only` modes, with camera + gallery input for the inspection image.

## Phase 1 — Backend prep (Python)

- [ ] Add `flask-cors` to `requirements.txt` and install it
- [ ] Wrap the Flask app with `CORS(app)` in `server.py` so requests from the Expo app's origin aren't blocked
- [ ] Confirm `app.run(host="0.0.0.0", ...)` — **already done**, no change needed
- [ ] Decide whether to keep `debug=True` for the demo or turn it off for a more stable long-running session (debug reloader can restart mid-demo)
- [ ] Confirm `MAX_CONTENT_LENGTH` (currently 32MB) is enough for 5 reference images + 1 test image at phone-camera resolution — test with real photos
- [ ] Sanity-check `.env` has a valid `GEMINI_API_KEY` and `GEMINI_MODEL` before any mobile testing starts

## Phase 2 — ngrok setup

- [ ] Sign up at ngrok.com, install the CLI, run `ngrok config add-authtoken`
- [ ] Start Flask (`python server.py`) and `ngrok http 5000` together, confirm the public URL reaches `/api/inspect` (test with `curl` or Postman first, before involving the mobile app)
- [ ] Note: free-tier ngrok URL changes every restart — this is exactly why the app has a settings screen instead of a hardcoded URL

## Phase 3 — Expo project setup

- [ ] Install Expo CLI tooling (`npx create-expo-app`)
- [ ] Install dependencies: `expo-image-picker` (camera + gallery), `@react-native-async-storage/async-storage` (persist server URL)
- [ ] Set up basic navigation (2 screens: Main, Settings — a simple stack or even a modal is enough for a demo)
- [ ] Confirm the app runs in **Expo Go** on your Android phone via QR code

## Phase 4 — Main screen UI

- [ ] Task description text input (maps to `task` form field, required)
- [ ] Mode toggle: **With reference images** / **Prompt-only** (maps to `mode` = `with_references` | `prompt_only`)
- [ ] Reference images section — **only visible when mode = with_references**:
- [ ] "Add reference images" button → gallery multi-select

- [ ] Enforce max of 5 (matches `MAX_REFERENCE_IMAGES` from `inspector.py`)
- [ ] Thumbnail previews with a remove (×) option per image
- [ ] Inspection image section — **always visible**:
- [ ] Single button offering "Take Photo" or "Choose from Gallery" (`expo-image-picker` supports both from one flow)
- [ ] Thumbnail preview + remove option
- [ ] "Run inspection" button:
- [ ] Disabled/loading state while request is in flight
- [ ] Client-side validation matching `server.py`'s checks: task required, test image required, exactly 1 test image, ≤5 reference images
- [ ] Match the existing UX nudge from `app.js`: if mode is `with_references` and 0 reference images are added, warn the user rather than silently running unanchored

## Phase 5 — API integration

- [ ] Build the multipart form POST to `<server_url>/api/inspect` with fields: `task`, `mode`, `test_image` (1 file), `reference_images` (0–5 files, only when mode is with_references) — field names must match exactly
- [ ] Read the server URL from AsyncStorage (set via Settings screen) as the request base — never hardcode it
- [ ] Handle the response shape from `server.py`'s `run_record`: `run_id`, `timestamp`, `task`, `mode`, `model`, `reference_count`, `results[0]` (`filename`, `verdict`, `finding`, `assessment`, `location`, `defect_percentage`, `confidence_score`, `reasoning`, `error`, `timing`), `metrics`, `timing_summary`
- [ ] Handle non-200 responses (`{"error": "..."}` shape) — task missing, bad mode, too many images, unsupported file type, missing API key
- [ ] Handle network-level failures (ngrok tunnel down, wrong URL pasted, request timeout) with a clear on-screen message, not a silent failure

## Phase 6 — Results screen UI

- [ ] Verdict badge — 4 states: `conforming` (pass) / `defective` (fail) / `uncertan` (uncertain) / `error`, styled distinctly per state
- [ ] Finding, Assessment, Location — text blocks
- [ ] Defect percentage + confidence score (`confidence_score` is 0.0–1.0 in this API version — display as a percentage, e.g. `×100`)
- [ ] Reasoning — text block
- [ ] Thumbnail of the inspected image
- [ ] Optional (nice-to-have, matches web version): metrics panel when filename starts with `good_`/`defect_` and `metrics.labeled_count > 0`
- [ ] Optional (nice-to-have): timing panel from `timing` + `timing_summary`
- [ ] Error state: show `result.error` clearly when `verdict === "error"`

## Phase 7 — Settings screen

- [ ] Text input for server (ngrok) URL

- [ ] Save to AsyncStorage on submit, load on app start
- [ ] Simple validation (starts with `http`) before saving
- [ ] Show current saved URL so it's obvious what the app is pointed at

## Phase 8 — Testing

- [ ] Test `prompt_only` mode end-to-end on a real photo
- [ ] Test `with_references` mode with 1, 3, and 5 reference images
- [ ] Test the >5 reference images rejection (client-side and server-side)
- [ ] Test missing task / missing test image validation
- [ ] Test camera capture path and gallery-picker path separately
- [ ] Test behavior when ngrok URL is stale/wrong (should fail gracefully)
- [ ] Test a `good_*`/`defect_*` filename to confirm metrics render (if built)

## Phase 9 — Build & install (installable APK)

- [ ] Set up EAS Build (`eas build -p android`) — cloud build, no local Android Studio needed
- [ ] Configure `eas.json` for a debug/preview APK profile (faster, simpler than a production build for a demo)
- [ ] Download and install the generated `.apk` directly on your Android phone
- [ ] Confirm the installed (non-Expo-Go) build still works against a fresh ngrok URL entered via Settings

## Phase 10 — Polish (optional, time-permitting)

- [ ] App name + icon for the installed app
- [ ] Loading state copy matching mode (e.g. "Comparing against references…" vs "Inspecting part…", mirroring `app.js`'s existing behavior)
- [ ] Basic empty-state message before first run
- [ ] Visual pass to match the existing web app's look/feel if desired